VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO5085 EXERCISES REPORT

DEEP LEARNING AND ITS APPLICATIONS IN COMPUTER VISION

Group: Traiblazer

Instructors: Dr. Le Thanh Sach
Dr. Nguyen Quoc Minh

Student: Dinh Hoang Duy Khanh 2670306

Ho Chi Minh City, November 2026

Member List and Workload

No.	Name	Student ID	Main responsibilities	Contribution
1	Dinh Hoang Duy Khanh	2670306	Build and run E1–E3, create figures, verify results, prepare the GitHub page, and write this report.	100%

Contents

Member List and Workload **2**

Introduction **4**

Shared Learning Workflow and Notation 4

1 E1 – Softmax, MLP, and CNN **6**

1.1 Objective and model design 6

1.2 E1 Theory and Code Workflow 6

1.3 Results 7

1.4 Error analysis 8

2 E2 – Multi-Head Self-Attention for Images **10**

2.1 Tokens and attention 10

2.2 Implemented attention architecture 10

2.3 Scaled Dot-Product Attention 11

3 E3 – LSTM and GRU for Image Sequences **15**

3.1 Image as a sequence 15

3.2 LSTM and GRU Gates 16

3.3 Interpretation 18

4 Requirement Checklist and Conclusion **19**

References **20**

Introduction

This report completes Exercises E1, E2, and E3 of CO5085 with one shared small dataset: Fashion-MNIST [1]. Fashion-MNIST contains 28×28 grayscale clothing images in ten classes. It is allowed by the exercise sheet, visually more varied than handwritten digits, and small enough for a beginner to run on a personal computer.

The official 60,000 training images are shuffled with seed 42, then split into 48,000 training images and 12,000 validation images. The official 10,000 test images are kept for final evaluation. The same split is used in every exercise, so comparisons are fair. All models use Adam [2] with learning rate 0.001, batch size 256, and cross-entropy loss. E1 and E3 use three epochs. E2 uses two epochs because it contains three attention experiments. These short runs compare designs clearly; they do not claim a fully optimized benchmark score.

How to read the results. Validation accuracy is used during training to compare choices. Test accuracy is reported after an experiment completes. A parameter count is the number of learned values in a model. A confusion matrix shows which classes are confused instead of only showing one average score.

Reproducibility and AI-use declaration

The project uses direct PyTorch [3] code. Run `python -m E1.run_e1`, `python -m E2.run_e2`, and `python -m E3.run_e3` from the repository root. The first run downloads Fashion-MNIST. Figures and JSON metrics are saved in `reports/`. AI assistance was used to help organize code and explain concepts. The student reviewed the code, ran the experiments, and is responsible for understanding and defending every model, result, and conclusion in this report.

Shared Learning Workflow and Notation

One image is a tensor $x \in \mathbb{R}^{1\times28\times28}$. The first dimension is one grayscale channel, and the other two dimensions are image height and width. The loader changes raw pixel values from 0–255 into floating-point values, then normalizes them. A mini-batch contains $B = 256$ images and labels. A label y is an integer from 0 to 9; the model produces ten scores, one score for each clothing class.

Softmax, Cross-Entropy, and Gradient Update

Let $z = (z_1, \ldots, z_{10})$ be the ten raw scores, also called logits. Softmax converts these scores into class probabilities:

$$p_i = \frac{\exp(z_i)}{\sum_{j=1}^{10} \exp(z_j)}.$$

Here p_i is the predicted probability of class i. A larger logit gives a larger probability, and all ten probabilities sum to one. For a true class index y, cross-entropy loss is

$$\mathcal{L} = -\log(p_y).$$

The loss is small when the true class receives high probability and large when it receives low probability. In the code, `nn.CrossEntropyLoss()` receives logits directly and performs the stable softmax-and-loss calculation internally. Backpropagation computes the gradient $\nabla_\theta \mathcal{L}$ for every learned parameter θ. Adam then updates the parameters; the basic gradient-descent idea is

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L},$$

where $\eta = 0.001$ is the learning rate. Adam uses extra running averages to make this update more stable, but the direction still comes from the gradient.

Algorithm 1: Shared Training and Evaluation Loop

The following steps match `get_loaders`, `train_epoch`, and `evaluate` in `common.py`.

1. Set the random seed to 42, load Fashion-MNIST, and recreate the same 48,000/12,000 train-validation indices.
2. Read one batch of normalized images and integer labels, then move both tensors to the selected device.
3. Call the model with the image batch. This is the forward pass and returns ten logits per image.
4. Compute cross-entropy loss from logits and true labels.
5. During training, clear old gradients, run backpropagation, then update parameters. The code calls `zero_grad()`, `backward()`, and `step()` in that order.
6. Count predictions with `argmax`; accuracy is $\frac{\text{number of correct predictions}}{\text{number of images}}$.
7. At the end of each epoch, switch to `model.eval()` and use `torch.no_grad()` for validation. This measures the model without changing weights.
8. Keep the official test set separate until the experiment has finished, then report final test accuracy and save JSON and figures.

1 E1 – Softmax, MLP, and CNN

1.1 Objective and model design

E1 requires three classifiers and a custom training loop. The shared loop in `common.py` is explicit: for each batch it predicts scores, computes cross-entropy loss, calls `loss.backward()`, then calls `optimizer.step()`. This makes the learning process visible instead of hiding it inside a high-level trainer.

The Softmax classifier flattens an image into 784 pixel values and uses one linear layer with ten outputs. The MLP also flattens the image, then uses hidden layers of 128 and 64 values with ReLU activation. The CNN keeps the image grid. It applies convolution, ReLU, and max-pooling twice before a small fully connected head [4]. Convolution helps the CNN learn local edges and clothing shapes from nearby pixels.

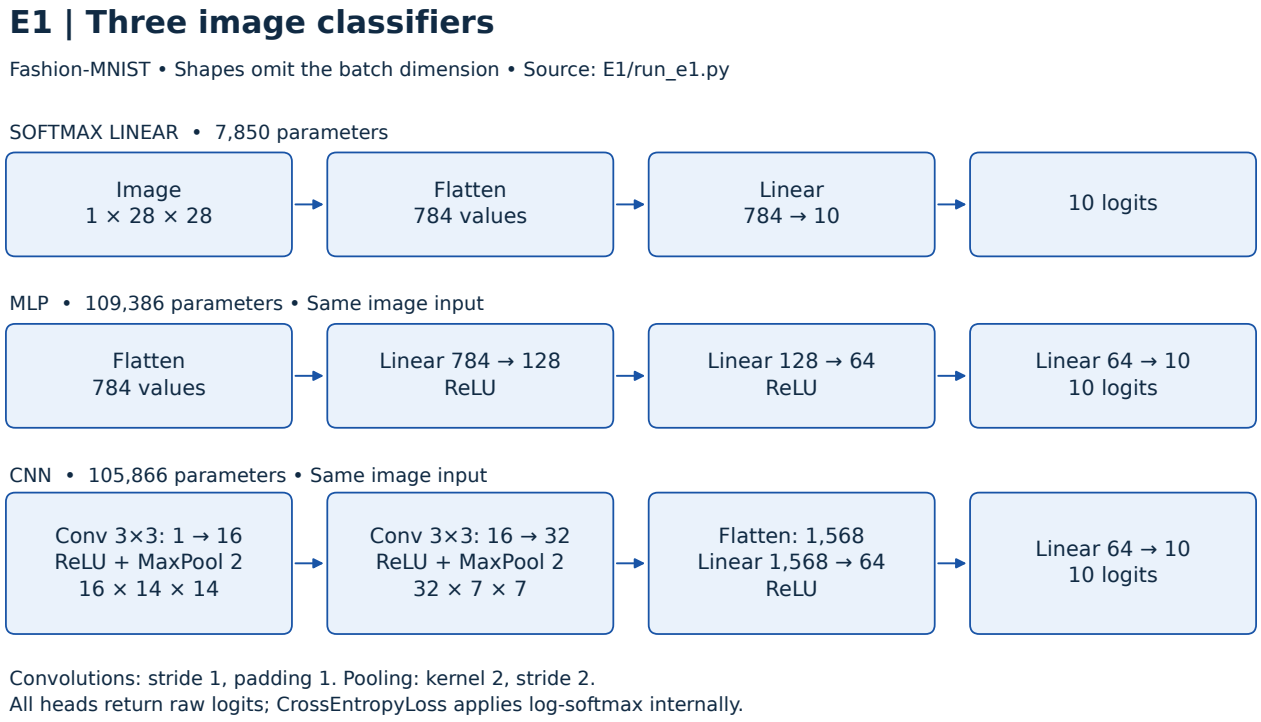


Figure 1.1: Implemented E1 architectures, drawn from `E1/run_e1.py`. Shapes omit the batch dimension. All classifiers return logits directly to cross-entropy loss.

1.2 E1 Theory and Code Workflow

For the linear Softmax classifier, flattening gives $v \in \mathbb{R}^{784}$. The model computes $z = vW + b$, where $W \in \mathbb{R}^{784\times10}$ is a learned weight matrix and $b \in \mathbb{R}^{10}$ is a learned bias. This model can only make one linear decision boundary for each class.

The MLP adds nonlinear layers:

$$h_1 = \mathrm{ReLU}(vW_1 + b_1), \qquad h_2 = \mathrm{ReLU}(h_1W_2 + b_2), \qquad z = h_2W_3 + b_3.$$

ReLU is $\mathrm{ReLU}(a) = \max(0, a)$. It keeps positive values and sets negative values to zero. Because of ReLU, the MLP can represent patterns that a single linear layer cannot.

For a CNN, a filter reads a small local window. At position (r, c), one output value can be written as

$$a_{r,c,k} = \sum_{u,v,d} x_{r+u,c+v,d}\, w_{u,v,d,k} + b_k.$$

The indices u, v move inside the small kernel, d selects an input channel, and k selects a learned filter. ReLU follows the convolution. Max-pooling keeps the largest value in a small region, reducing height and width while preserving a strong detected feature. In `CNNClassifier`, two convolution-and-pooling blocks turn the image into learned feature maps before the final classifier.

The E1 script creates one model at a time, calls the shared loop for three epochs, records loss and validation accuracy after each epoch, evaluates the official test set once, then saves a confusion matrix and several incorrect CNN predictions. Thus the displayed error examples come from real final predictions, not selected images.

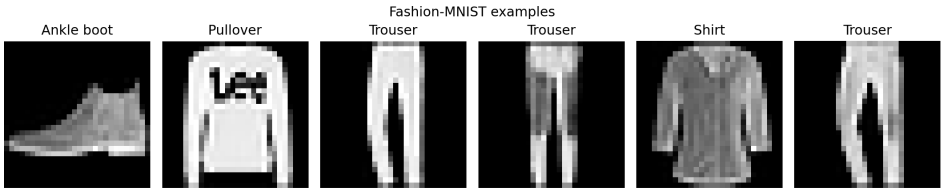


Figure 1.2: Examples from Fashion-MNIST. Each image has one clothing label.

1.3 Results

Table 1.1: E1 comparison on the same test set. Time includes training and evaluation.

Model	Parameters	Time (s)	Test accuracy
Softmax linear	7,850	125.2	0.8258
MLP	109,386	103.2	0.8518
CNN	105,866	156.4	**0.8686**

The CNN is best at 0.8686. It has nearly the same number of parameters as the MLP, so the improvement is not simply from a larger model. The CNN keeps local two-dimensional relationships; the MLP loses them when it flattens the image.

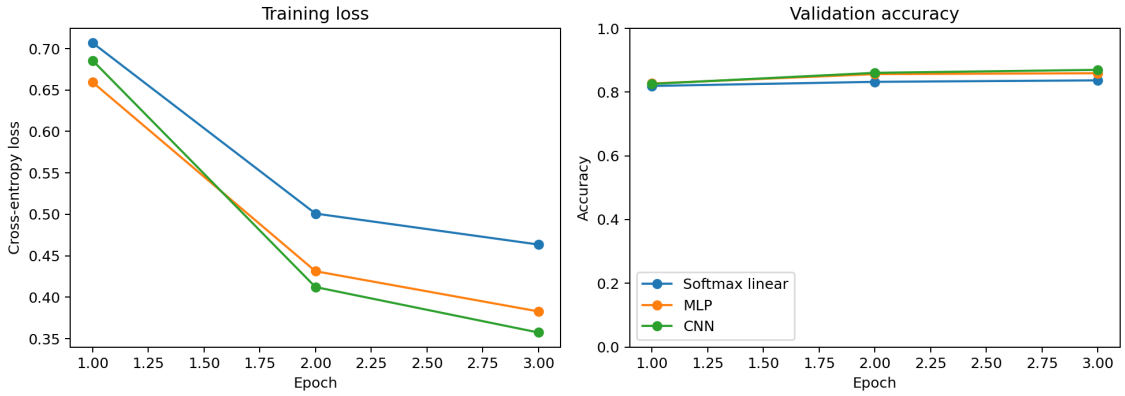


Figure 1.3: Training loss and validation accuracy for E1.

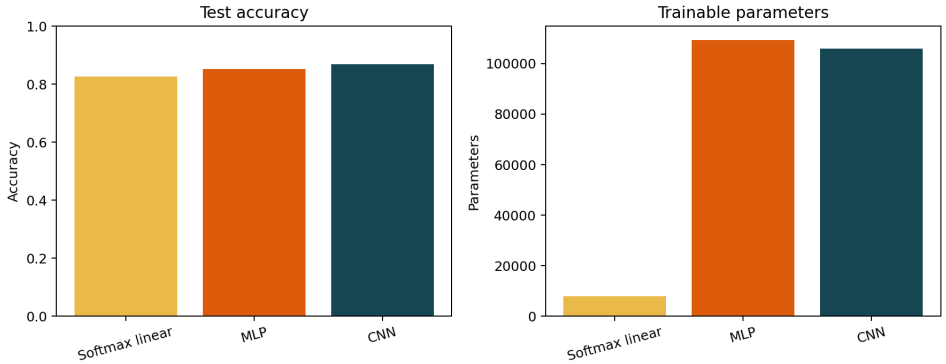


Figure 1.4: E1 test accuracy and parameter comparison.

1.4 Error analysis

The CNN confusion matrix in Figure 1.5 is calculated from the final test predictions. The diagonal contains correct predictions. Most off-diagonal errors are between visually similar upper-body classes such as shirt, T-shirt/top, pullover, and coat. This is reasonable because the images are grayscale and low resolution. Figure 1.6 shows individual errors, which explains the score better than accuracy alone.

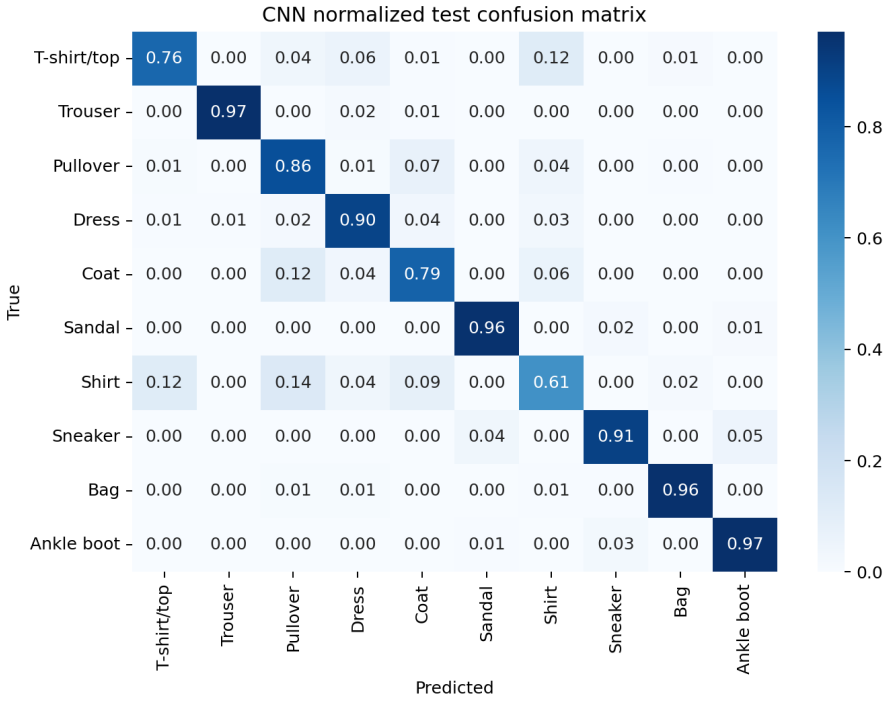


Figure 1.5: Confusion matrix for the selected E1 CNN.

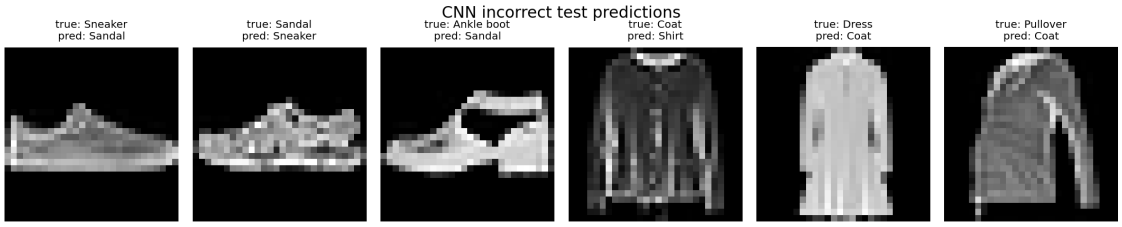


Figure 1.6: A small selection of CNN errors. Titles give true and predicted labels.

2 E2 – Multi-Head Self-Attention for Images

2.1 Tokens and attention

Attention first needs a sequence of image tokens. The patch tokenizer cuts an image into 49 non-overlapping 4×4 patches and projects each patch to 32 values, drawing on Vision Transformer tokenization [5]. The CNN-stem tokenizer uses two small convolution-and-pooling stages, then converts its 7×7 feature map into 49 tokens. The sequence length is the same, but the CNN stem supplies tokens that already contain local visual features. Additionally, the codebase implements a third representation, `RowTokenizer`, converting each 28-pixel row into a token and directly connecting with the sequential image modeling in E3.

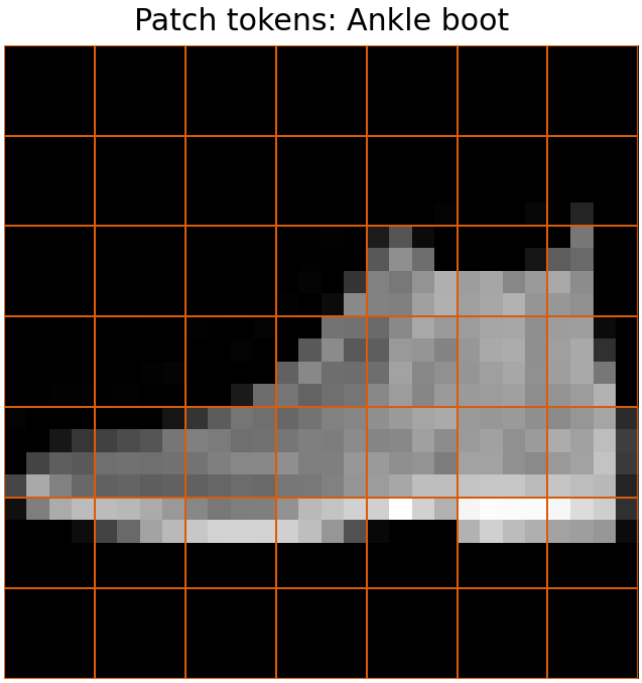


Figure 2.1: One image divided into 49 patch tokens.

The manual MSA uses only linear layers, matrix multiplication, transpose, softmax, and reshape, following the scaled dot-product attention formulation [6]. For tokens X, it forms $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. Attention weights are $\text{softmax}(QK^T/\sqrt{d})$ and the output is a weighted sum of V. The code splits 32 values into two heads, computes each head, and joins them. It does not call `nn.MultiheadAttention`. The next experiment uses PyTorch MSA with the same patch tokens and settings. The third experiment changes only the tokenizer to the CNN stem.

2.2 Implemented attention architecture

Figure 2.2 shows the complete classifier, including learned positions, both residual connections, post-attention LayerNorm, the feed-forward network, and mean pooling. The measured

E2 runs are manual MSA with patch tokens, PyTorch MSA with patch tokens, and PyTorch MSA with CNN-stem tokens. Row tokenization is implemented and shape-tested, but is not one of the three measured runs in `e2_results.json`.

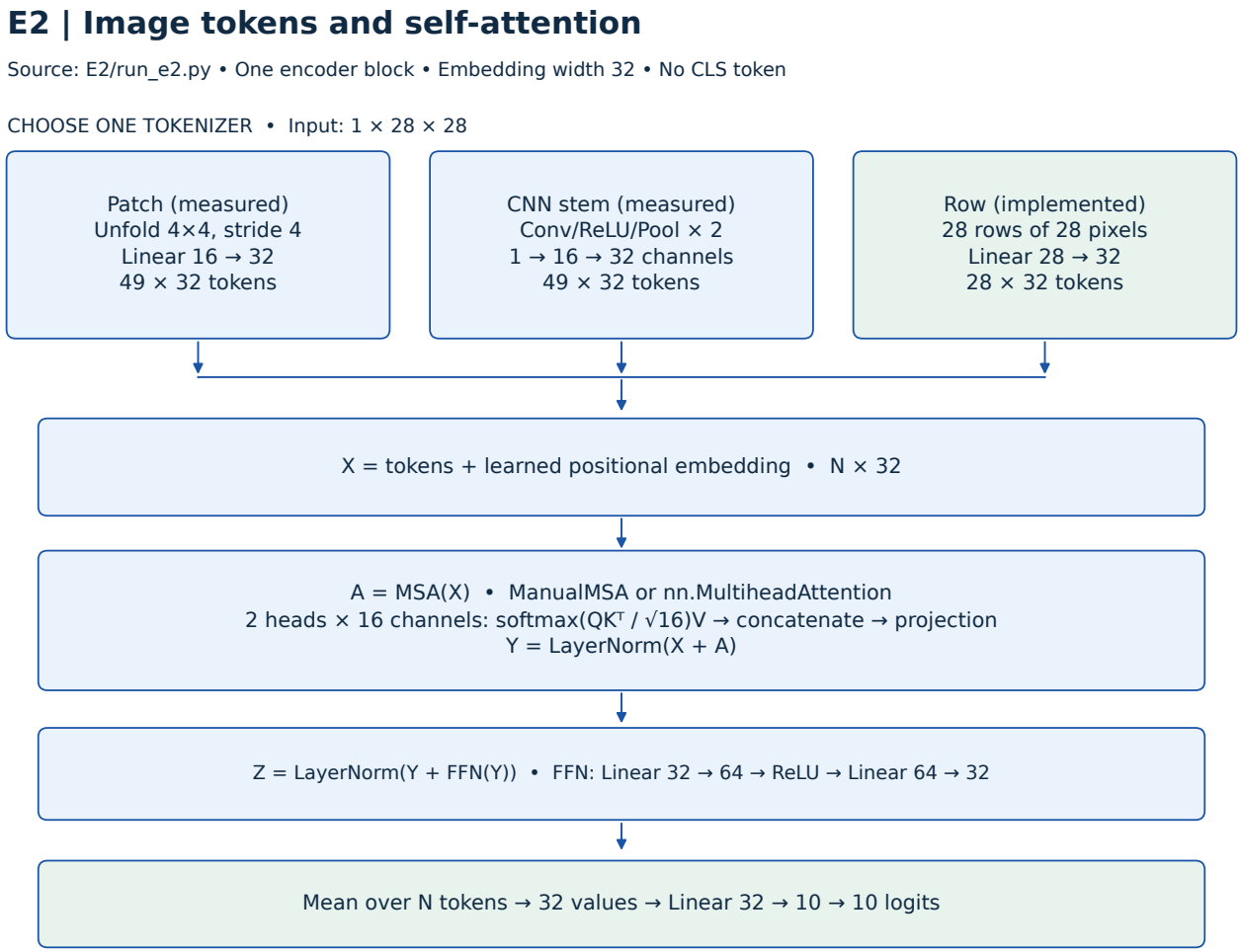


Figure 2.2: E2 tokenizers and the single attention block, matching `AttentionClassifier`. $N = 49$ for patch/CNN-stem tokens and $N = 28$ for row tokens. The model uses two heads of width 16 and no class token.

2.3 Scaled Dot-Product Attention

For an image token matrix $X \in \mathbb{R}^{49\times32}$, each row is one of 49 tokens and each token has 32 values. The model uses three learned projections:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

Query Q asks what information a token needs. Key K describes what a token offers. Value V is the information that can be passed to another token. For one attention head with head

dimension $d_h = 16$, the scaled dot-product result is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_h}}\right)V.$$

The matrix QK^T compares every token with every other token. Dividing by $\sqrt{d_h}$ prevents large dot products from making softmax too sharp. Each row after softmax contains weights that sum to one, so the result is a weighted combination of value vectors.

2.3.1 Algorithm 2: Manual Multi-Head Self-Attention

This algorithm follows `ManualMSA.forward` in `E2/run_e2.py`.

1. Create tokens using either 49 flattened 4×4 patches or 49 positions from the CNN feature map.
2. Apply four linear layers to compute Q, K, V, and the final output projection.
3. Reshape Q, K, and V from 32 values into two heads of 16 values using `split_heads`.
4. For every head, compute $QK^T/\sqrt{16}$, apply softmax across keys, and multiply the weights by V.
5. Transpose and reshape the two heads back into 32 values, then apply the output linear layer.
6. Add the original tokens to the attention result, normalize with `LayerNorm`, and apply the small feed-forward network with a second residual connection.
7. Average the final 49 tokens and apply one linear classifier to obtain ten logits.

The PyTorch comparison calls `nn.MultiheadAttention` at the same point in the model. Matching patch-token results are evidence that the manual shapes, scaling, and head merge are working as intended. The CNN-stem experiment changes the input representation only, making the tokenization comparison meaningful.

Table 2.1: E2 attention and tokenization comparison; all runs use two epochs.

Model	Parameters	Time (s)	Test accuracy
Manual MSA + patch	10,986	110.7	0.7061
PyTorch MSA + patch	10,986	95.5	0.7072
PyTorch MSA + CNN stem	15,242	119.1	**0.7578**

Manual and PyTorch MSA with patch tokens have almost the same accuracy, 0.7061 and 0.7072. This is expected because their tokenization and architecture are matched. PyTorch

MSA is a little faster. The CNN-stem tokenizer improves test accuracy to 0.7578 because it adds local edge and texture features before attention. The attention models remain below the E1 CNN because they are shallow and trained for only two epochs without augmentation or pretraining.

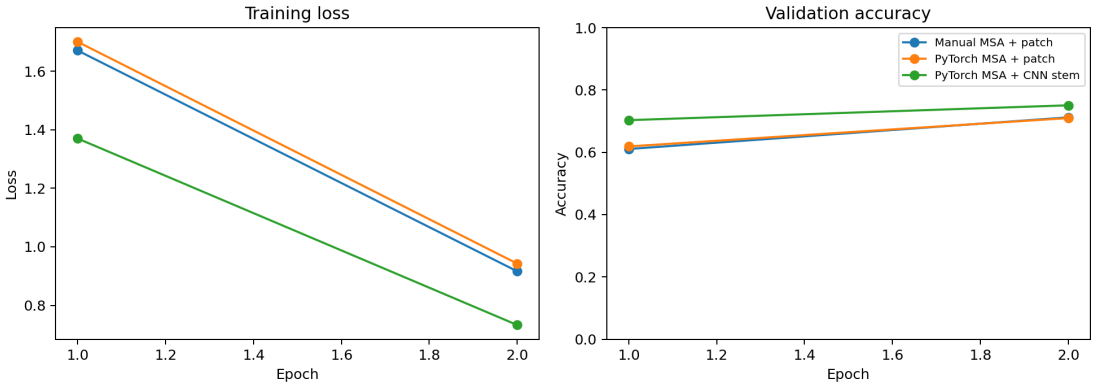


Figure 2.3: E2 learning curves.

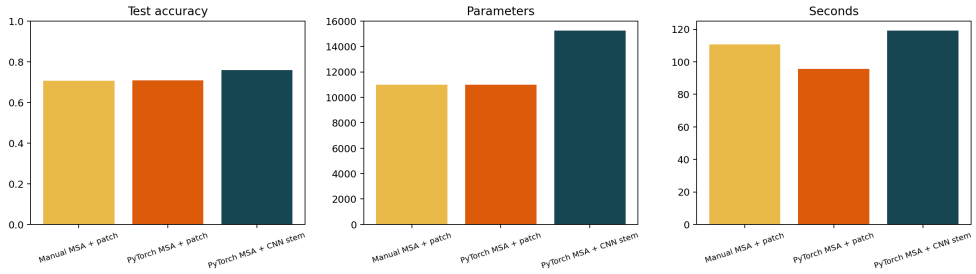


Figure 2.4: E2 comparison of accuracy, parameters, and time.

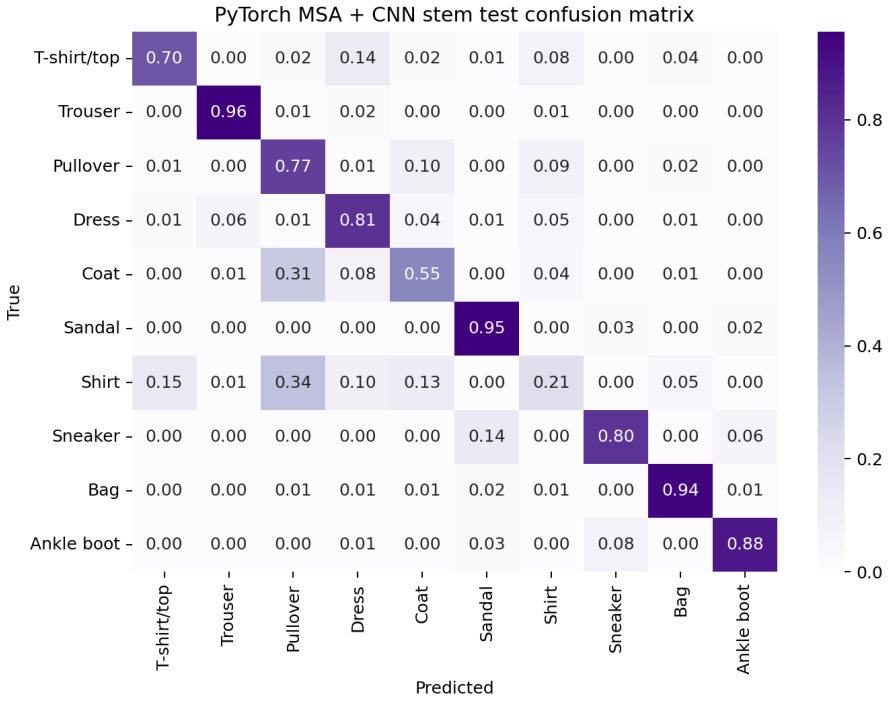


Figure 2.5: Confusion matrix for PyTorch MSA with CNN-stem tokens.

3 E3 – LSTM and GRU for Image Sequences

3.1 Image as a sequence

E3 reads an image as a sequence instead of a grid. Each of the 28 image rows is one time step, and every step contains 28 normalized pixels. The sequence therefore has length 28 and input size 28. This is simple to implement, but it has a limitation: a recurrent model must compress earlier rows into a hidden state and does not naturally inspect nearby pixels in both directions as a CNN does.

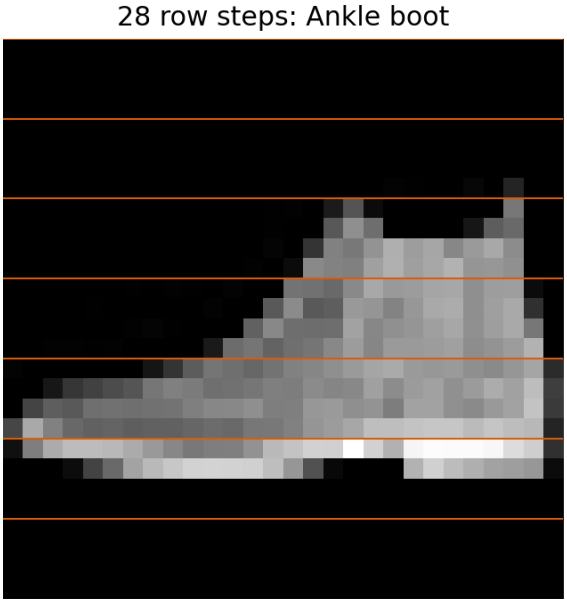


Figure 3.1: Rows marked as time steps for LSTM and GRU.

LSTM [7] and GRU [8] both use hidden size 64 and a final linear classifier. The last recurrent output predicts the clothing class. LSTM has separate gates and a cell state, so it has more parameters. GRU combines some gates, so it is smaller. The E1 CNN result is the image-grid baseline.

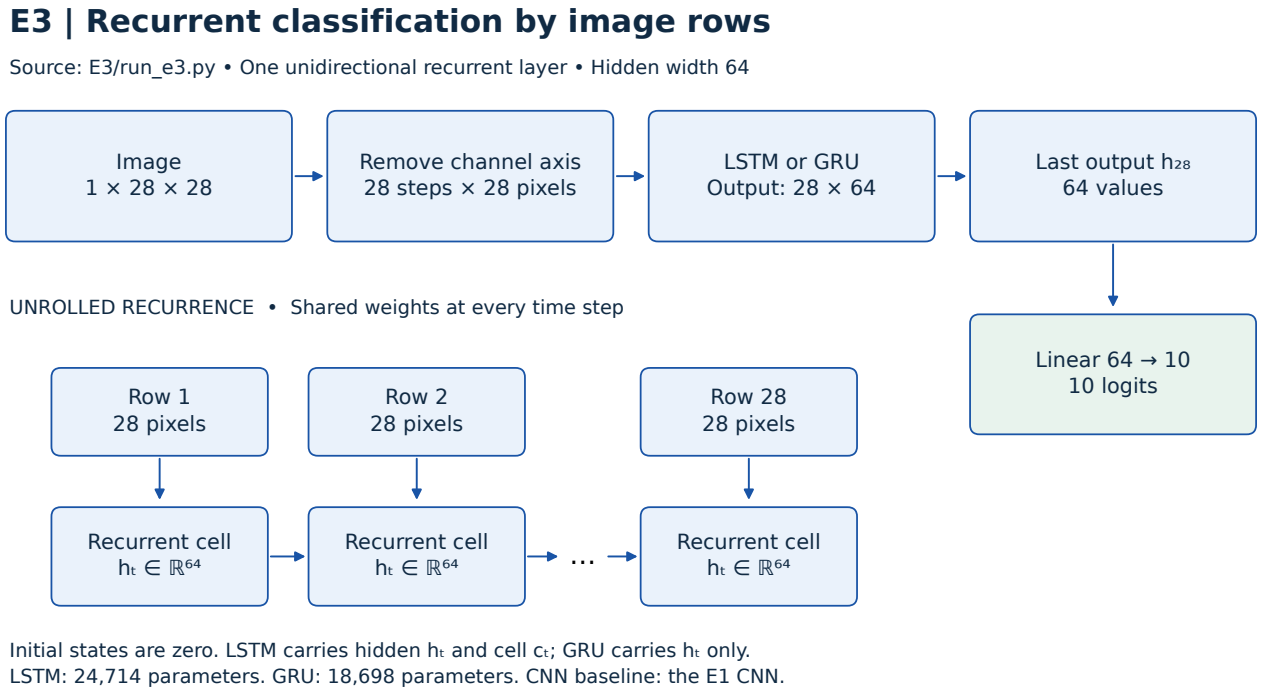


Figure 3.2: E3 LSTM and GRU architecture, drawn from `RowSequenceClassifier`. The classifier uses the last of 28 recurrent outputs, not an average of all time steps. The CNN baseline is the same architecture shown in Figure 1.1.

3.2 LSTM and GRU Gates

Let $r_t \in \mathbb{R}^{28}$ be row t of an image and h_{t-1} be the hidden state from the previous row. The LSTM controls what to keep and forget with gates:

$$\begin{aligned} i_t &= \sigma(W_i[r_t, h_{t-1}] + b_i), \quad & f_t &= \sigma(W_f[r_t, h_{t-1}] + b_f), \\ o_t &= \sigma(W_o[r_t, h_{t-1}] + b_o), \quad & g_t &= \tanh(W_g[r_t, h_{t-1}] + b_g), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, \quad & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

The sigmoid function σ gives a value between 0 and 1, so gates can act like soft switches. The symbol $\odot$ means element-by-element multiplication. The cell state c_t carries information through many steps; the hidden state h_t is the output used at the next step.

GRU uses fewer gates. The following equations match the `nn.GRU` implementation used in E3 [9]; q_t denotes the reset gate so that r_t can continue to denote an image row:

$$\begin{aligned} q_t &= \sigma(W_{iq} r_t + b_{iq} + W_{hq} h_{t-1} + b_{hq}), \\ z_t &= \sigma(W_{iz} r_t + b_{iz} + W_{hz} h_{t-1} + b_{hz}), \\ \widetilde{h}_t &= \tanh(W_{in} r_t + b_{in} + q_t \odot (W_{hn} h_{t-1} + b_{hn})), \\ h_t &= (1 - z_t) \odot \widetilde{h}_t + z_t \odot h_{t-1}. \end{aligned}$$

The update gate z_t controls how much of the previous hidden state is retained. In PyTorch, the reset gate multiplies the transformed hidden-state term, including its bias. This placement differs from applying the reset gate to h_{t-1} before the matrix multiplication. The smaller GRU structure explains its lower parameter count and shorter measured runtime.

3.2.1 Algorithm 3: Row-Sequence Classification

The following steps match `RowSequenceClassifier` and `train_model` in `E3/run_e3.py`.

1. Start with an image batch of shape $(B, 1, 28, 28)$ and remove the single channel dimension.
2. Read the result as $(B, 28, 28)$: 28 row time steps, each containing 28 pixel values.
3. Send the full sequence to either `nn.LSTM` or `nn.GRU` with hidden size 64.
4. Select the last output h_{28} because it summarizes the rows processed from top to bottom.
5. Apply the final linear layer to get ten logits, then use the same cross-entropy and optimizer steps as Algorithm 1.
6. Compare LSTM and GRU with the saved CNN metric from E1. CNN is a fair baseline because it receives the original two-dimensional image rather than a row sequence.

Table 3.1: E3 recurrent models and the CNN baseline.

Model	**Parameters**	**Time (s)**	**Test accuracy**
LSTM rows	24,714	131.1	0.8201
GRU rows	18,698	102.6	0.8156
CNN baseline (E1)	105,866	156.4	**0.8686**

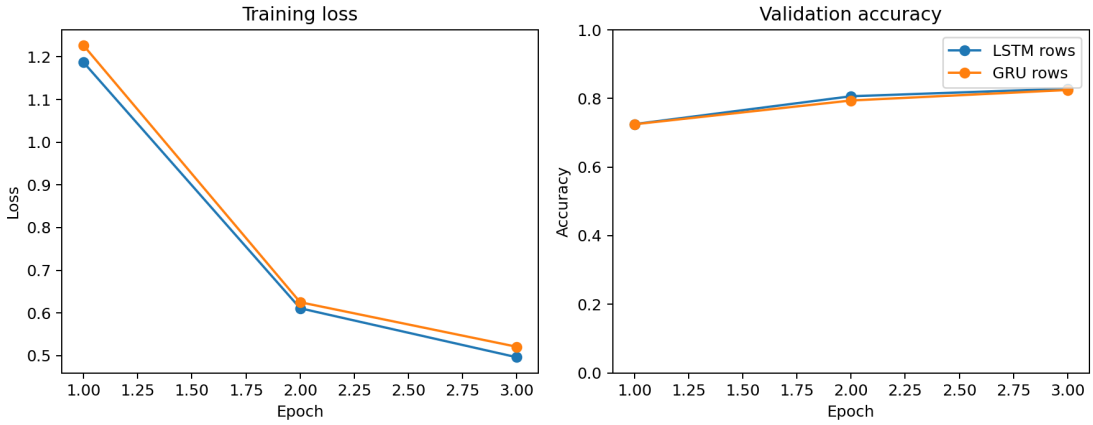


Figure 3.3: LSTM and GRU training loss and validation accuracy.

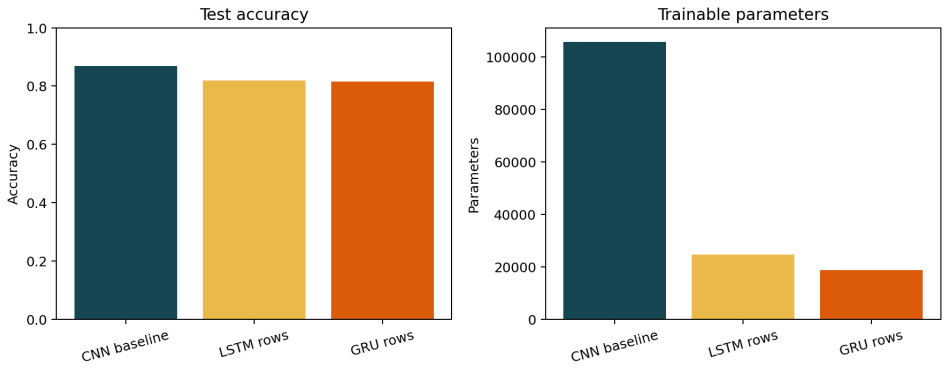


Figure 3.4: E3 accuracy, parameter, and time comparison.

3.3 Interpretation

LSTM gives 0.8201 test accuracy and GRU gives 0.8156. GRU is faster and has fewer parameters, while LSTM is slightly more accurate. Both are below the CNN at 0.8686. The result supports the representation argument: CNN sees nearby pixels in height and width, while the row-sequence model has to remember earlier rows in a hidden state. RNNs are natural for sequence data; for these small images, CNN is the better choice.

4 Requirement Checklist and Conclusion

Table 4.1: Evidence that the exercise requirements are completed.

Requirement	Evidence
One shared dataset	Fashion-MNIST and the same 48,000/12,000/10,000 split are used in E1–E3.
E1 models and errors	`E1/run_e1.py`, Table 1.1, curves, confusion matrix, and error samples.
Manual training loop	`common.py` explicitly runs forward, loss, backward, and optimizer step.
Manual and library MSA	`ManualMSA` uses matrix operations; the library comparison uses `nn.MultiheadAttention`.
Multiple tokenizations	Patch, CNN-stem, and row tokenizers in `E2/run_e2.py`; Table 2.1.
LSTM, GRU, baseline	`E3/run_e3.py` compares both recurrent units with the E1 CNN.
Reproducible delivery	Code, figures, JSON results, README, GitHub Pages source, and this PDF are stored together.

This project shows three ways to classify the same images. Softmax is the simplest baseline, the MLP learns nonlinear pixel combinations, and the CNN is strongest because it keeps the image layout. Manual attention agrees closely with PyTorch MSA, and CNN-stem tokens improve over raw patches. LSTM and GRU show how to read an image as a sequence, but CNN remains more suitable for this image dataset.

The code is intentionally short and direct. It shows the essential steps: prepare data, define a model, train with forward and backward propagation, evaluate a held-out test set, and inspect errors. Future work could add more epochs, augmentation, a deeper CNN, or a larger attention model. Those changes are outside this beginner-level submission.

References

[1] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," arXiv:1708.07747, 2017. https://github.com/zalandoresearch/fashion-mnist.

[2] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1412.6980.

[3] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. https://pytorch.org.

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[5] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2010.11929.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762.

[7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, 2014.

[9] PyTorch, "GRU: equations and implementation notes." https://docs.pytorch.org/docs/2.12/generated/torch.nn.GRU.html.